```
import pandas as pd
```

```
path="/content/drive/MyDrive/Dataset/Bike datas.xlsx"
df=pd.read_excel(path)
df.head(5)
```

	Date	Rented Bike Count	Hour	Temperature(蚓)	Humidity(%)	Wind speed (m/s)	Visibility (10m)	Dew point temperature(蚓)	Solar Radiation (MJ/m2)	Rainfall(mm)	Snowfall (cm)	Seaso
0	2017-12-01	254	0	-5.2	37	2.2	2000	-17.6	0.0	0.0	0.0	Win
1	2017-12-01	204	1	-5.5	38	0.8	2000	-17.6	0.0	0.0	0.0	Win
2	2017-12-01	173	2	-6.0	39	1.0	2000	-17.7	0.0	0.0	0.0	Win
3	2017-12-01	107	3	-6.2	40	0.9	2000	-17.6	0.0	0.0	0.0	Win
4	2017-	78	4	-6.0	36	2.3	2000	-18.6	0.0	0.0	0.0	Win

Next steps: Generate code with df New interactive sheet

Data Overview

Let's start by getting a general overview of the data, including its shape, data types, and non-null values.

```
print("DataFrame Shape:", df.shape)
print("\nDataFrame Info:")
df.info()
```

```
DataFrame Shape: (8760, 14)

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8760 entries, 0 to 8759
Data columns (total 14 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Date                       8760 non-null   datetime64[ns]
 1   Rented Bike Count          8760 non-null   int64
 2   Hour                       8760 non-null   int64
 3   Temperature(蚓)             8760 non-null   float64
 4   Humidity(%)                8760 non-null   int64
 5   Wind speed (m/s)           8760 non-null   float64
 6   Visibility (10m)           8760 non-null   int64
 7   Dew point temperature(蚓)   8760 non-null   float64
 8   Solar Radiation (MJ/m2)    8760 non-null   float64
 9   Rainfall(mm)               8760 non-null   float64
 10  Snowfall (cm)              8760 non-null   float64
 11  Seasons                    8760 non-null   object
 12  Holiday                    8760 non-null   object
 13  Functioning Day            8760 non-null   object
dtypes: datetime64[ns](1), float64(6), int64(4), object(3)
memory usage: 958.3+ KB
```

Missing Values

Next, let's check for any missing values in the dataset.

```
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Missing Values:
Date                       0
Rented Bike Count          0
Hour                       0
Temperature(蚓)             0
Humidity(%)                0
Wind speed (m/s)           0
```

```
Visibility (10m)            0
Dew point temperature(蚓)    0
Solar Radiation (MJ/m2)     0
Rainfall(mm)                0
Snowfall (cm)               0
Seasons                     0
Holiday                     0
Functioning Day             0
dtype: int64
```

Identify Numeric and Categorical Features

We can identify numeric and categorical features based on their data types.

```
numeric_features = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
categorical_features = df.select_dtypes(include=['object']).columns.tolist()

print("\nNumeric Features:", numeric_features)
print("\nCategorical Features:", categorical_features)
```

```
Numeric Features: ['Rented Bike Count', 'Hour', 'Temperature(蚓)', 'Humidity(%)', 'Wind speed (m/s)', 'Visibility (10m)', 'Dew p

Categorical Features: ['Seasons', 'Holiday', 'Functioning Day']
```

Sample Records

Finally, let's display a random sample of 5 records to get a feel for the data content.

```
print("\nSample Records (5 random rows):")
display(df.sample(5))
```

```
Sample Records (5 random rows):
```

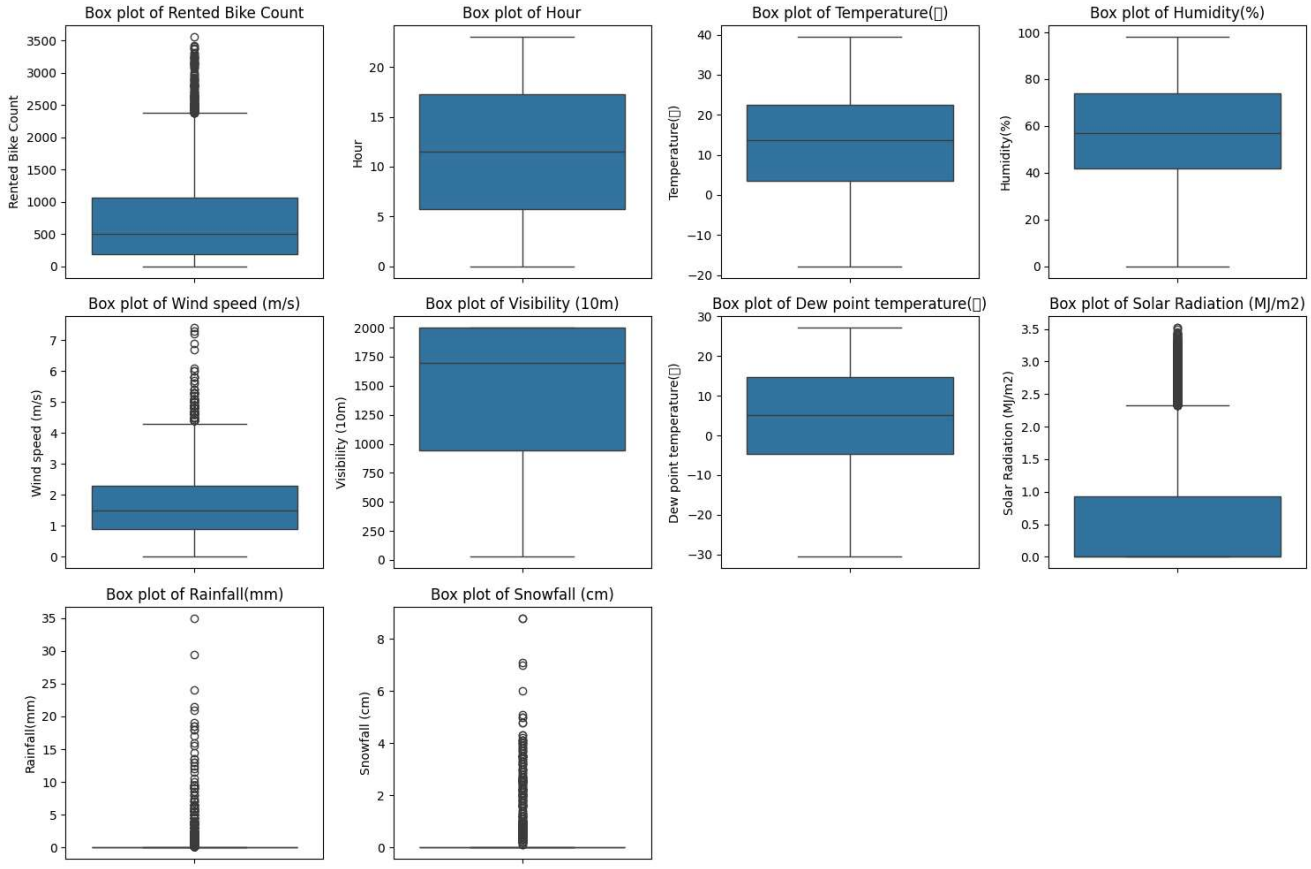
	Date	Rented Bike Count	Hour	Temperature(蚓)	Humidity(%)	Wind speed (m/s)	Visibility (10m)	Dew point temperature(蚓)	Solar Radiation (MJ/m2)	Rainfall(mm)	Snowfall (cm)	Se
7628	2018-10-14	1306	20	14.4	61	2.3	1945	6.9	0.00	0.0	0.0	A
1798	2018-02-13	291	22	2.6	57	3.1	2000	-5.0	0.00	0.0	0.0	
7208	2018-09-27	2129	8	14.8	51	1.1	2000	4.7	0.59	0.0	0.0	A
6778	2018-09-09	928	10	22.9	45	1.5	2000	10.3	2.10	0.0	0.0	A
8430	2018-	151	6	1.0	77	0.6	1259	-2.5	0.00	0.0	0.0	A

Outlier Detection and Handling

Let's visualize the distribution of numerical features using box plots to identify potential outliers. For simplicity, we'll use IQR-based capping to handle them.

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(15, 10))
for i, feature in enumerate(numeric_features):
    plt.subplot(3, 4, i + 1) # Adjust subplot grid as needed
    sns.boxplot(y=df[feature])
    plt.title(f'Box plot of {feature}')
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_1804/1218857019.py:9: UserWarning: Glyph 34451 (\N{CJK UNIFIED IDEOGRAPH-8693}) missing from font(s) DejaVu Sans.
  plt.tight_layout()
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 34451 (\N{CJK UNIFIED IDEOGRAPH-8693}
  fig.canvas.print_figure(bytes_io, **kw)
```



Some features, like `Rented Bike Count`, `Temperature(℃)`, `Solar Radiation (MJ/m2)`, `Rainfall(mm)`, and `Snowfall (cm)`, show outliers. Let's handle these using the Interquartile Range (IQR) method, capping values that fall outside 1.5 times the IQR.

```
for feature in ['Rented Bike Count', 'Temperature(蚓)', 'Solar Radiation (MJ/m2)', 'Rainfall(mm)', 'Snowfall (cm)']:
    Q1 = df[feature].quantile(0.25)
    Q3 = df[feature].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    df[feature] = df[feature].clip(lower=lower_bound, upper=upper_bound)

print("Outliers handled for selected numerical features using IQR capping.")
```

```
Outliers handled for selected numerical features using IQR capping.
```

Encoding Categorical Variables

We will use One-Hot Encoding for the categorical features: 'Seasons', 'Holiday', and 'Functioning Day'.

```
df_encoded = pd.get_dummies(df, columns=categorical_features, drop_first=True)

print("Categorical features encoded.")
display(df_encoded.head())
```

```
Categorical features encoded.
```

	Date	Rented Bike Count	Hour	Temperature(蚓)	Humidity(%)	Wind speed (m/s)	Visibility (10m)	Dew point temperature(蚓)	Solar Radiation (MJ/m2)	Rainfall(mm)	Snowfall (cm)	Seaso
0	2017-12-01	254.0	0	-5.2	37	2.2	2000	-17.6	0.0	0.0	0.0	
1	2017-12-01	204.0	1	-5.5	38	0.8	2000	-17.6	0.0	0.0	0.0	
2	2017-12-01	173.0	2	-6.0	39	1.0	2000	-17.7	0.0	0.0	0.0	
3	2017-12-01	107.0	3	-6.2	40	0.9	2000	-17.6	0.0	0.0	0.0	
4	2017-12-01	78.0	4	-6.0	36	2.3	2000	-18.6	0.0	0.0	0.0	

Feature Standardization

Finally, we will standardize the numerical features using `StandardScaler` from `sklearn.preprocessing`. We need to re-identify the numerical features after one-hot encoding, as new columns have been added.

```
from sklearn.preprocessing import StandardScaler

# Re-identify numerical features, excluding the 'Date' column
current_numeric_features = df_encoded.select_dtypes(include=['int64', 'float64']).columns.tolist()
if 'Date' in current_numeric_features:
    current_numeric_features.remove('Date')

scaler = StandardScaler()
df_encoded[current_numeric_features] = scaler.fit_transform(df_encoded[current_numeric_features])

print("Numerical features standardized.")
display(df_encoded.head())
```

```
Numerical features standardized.
```

	Date	Rented Bike Count	Hour	Temperature(蚓)	Humidity(%)	Wind speed (m/s)	Visibility (10m)	Dew point temperature(蚓)	Solar Radiation (MJ/m2)	Rainfall(mm)	Snowf (
0	2017-12-01	-0.709536	-1.661325	-1.513957	-1.042483	0.458476	0.925871	-1.659605	-0.682824	0.0	
1	2017-12-01	-0.789297	-1.516862	-1.539074	-0.993370	-0.892561	0.925871	-1.659605	-0.682824	0.0	
2	2017-12-01	-0.838748	-1.372399	-1.580936	-0.944257	-0.699556	0.925871	-1.667262	-0.682824	0.0	
3	2017-12-01	-0.944033	-1.227936	-1.597680	-0.895144	-0.796059	0.925871	-1.659605	-0.682824	0.0	
4	2017-12-01	-0.990294	-1.083473	-1.580936	-1.091596	0.554978	0.925871	-1.736177	-0.682824	0.0	

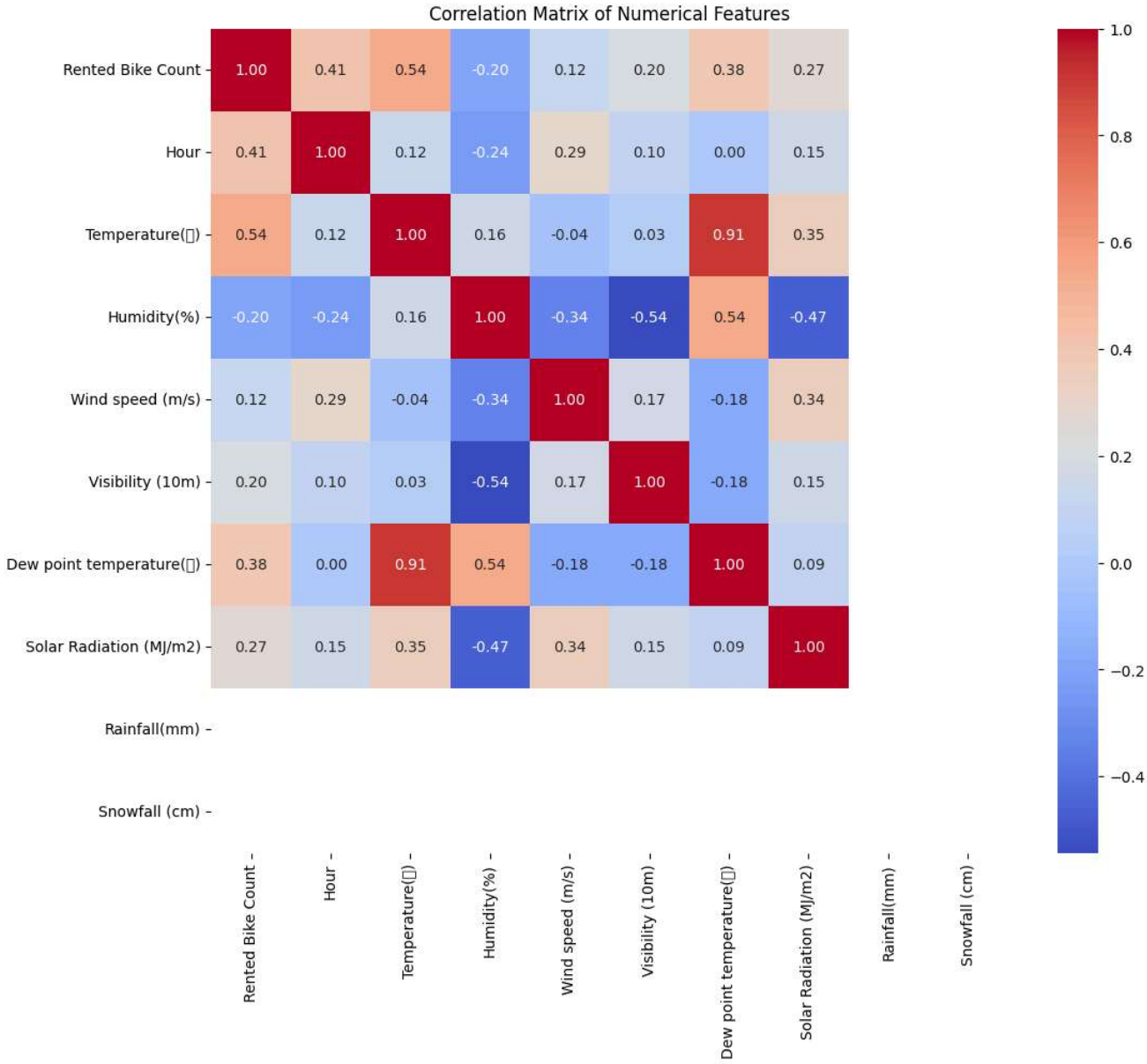
Correlation Matrix

Let's visualize the correlation matrix of the numerical features to understand their linear relationships.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Drop 'Date' column if it's still present in current_numeric_features for correlation
```

```
# current_numeric_features was already updated to exclude 'Date' in the previous cell.
correlation_matrix = df_encoded[current_numeric_features].corr()

plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix of Numerical Features')
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/seaborn/utils.py:61: UserWarning: Glyph 34451 (\N{CJK UNIFIED IDEOGRAPH-8693}) missing f
  fig.canvas.draw()
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 34451 (\N{CJK UNIFIED IDEOGRAPH-8693}
  fig.canvas.print_figure(bytes_io, **kw)
```

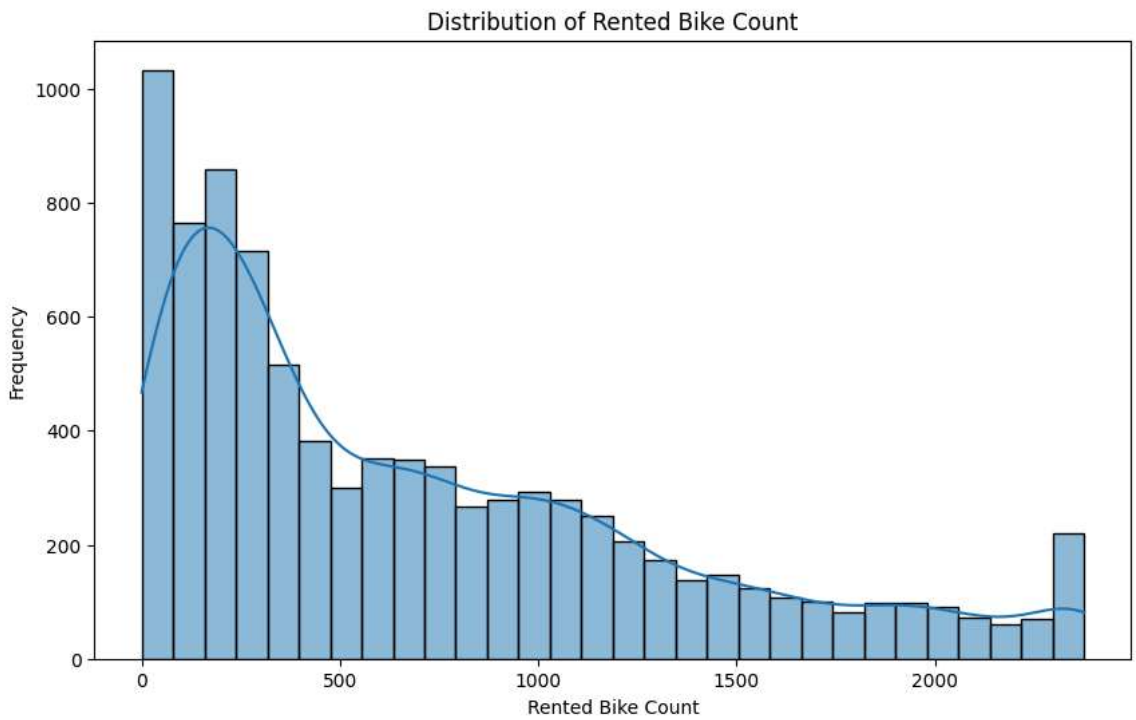


Data Visualization: Distributions and Relationships

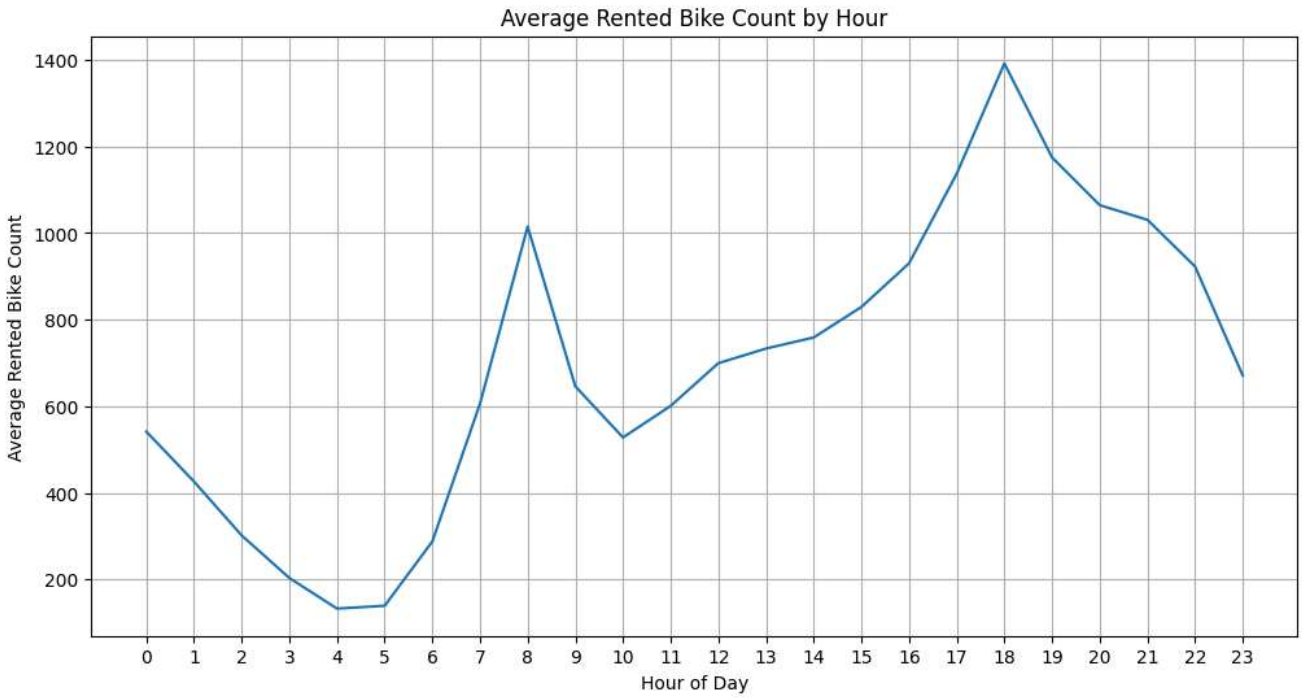
Now, let's create various plots to understand the distribution of key variables and their relationships with 'Rented Bike Count'.

```
# 1. Distribution of 'Rented Bike Count'
plt.figure(figsize=(10, 6))
sns.histplot(df['Rented Bike Count'], kde=True, bins=30)
plt.title('Distribution of Rented Bike Count')
plt.xlabel('Rented Bike Count')
```

```
plt.ylabel('Frequency')
plt.show()
```



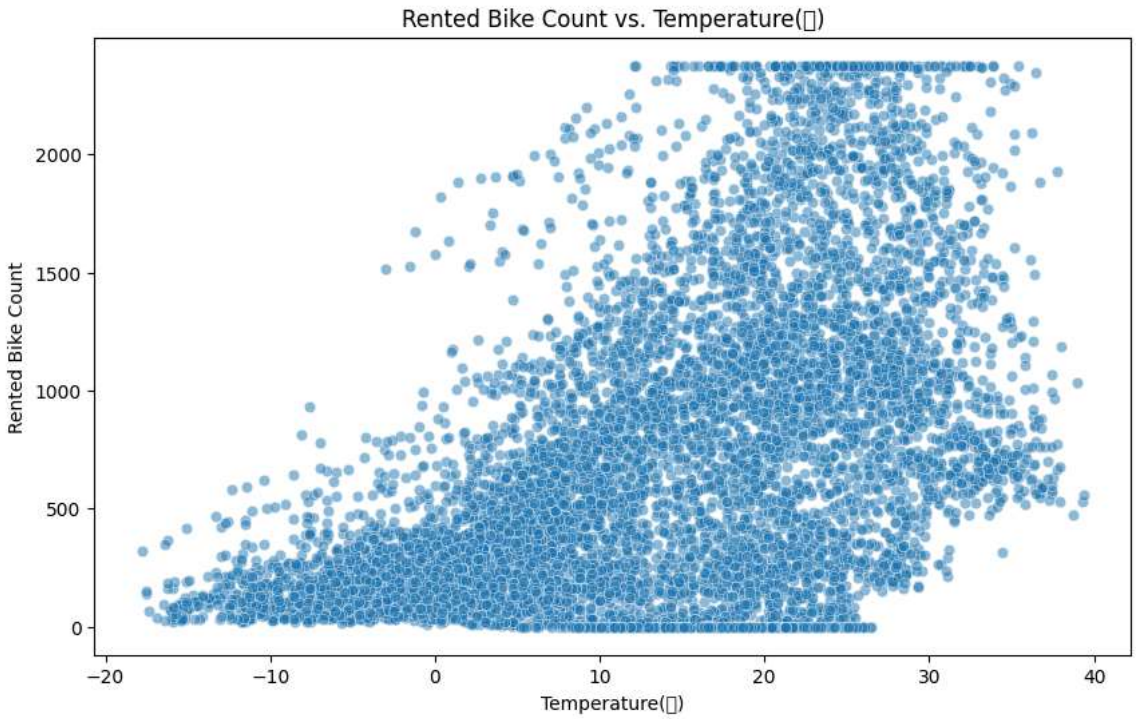

```
# 2. Rented Bike Count vs. Hour
plt.figure(figsize=(12, 6))
sns.lineplot(x='Hour', y='Rented Bike Count', data=df.groupby('Hour')['Rented Bike Count'].mean().reset_index())
plt.title('Average Rented Bike Count by Hour')
plt.xlabel('Hour of Day')
plt.ylabel('Average Rented Bike Count')
plt.xticks(range(0, 24))
plt.grid(True)
plt.show()
```



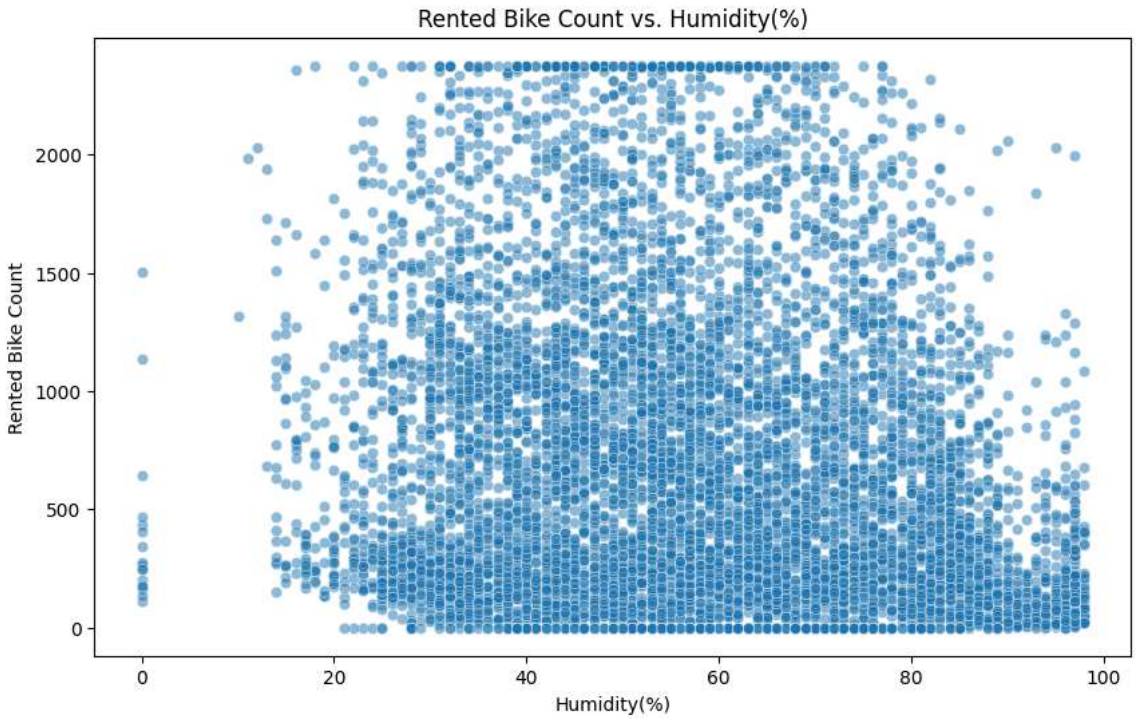

```
# 3. Rented Bike Count vs. Temperature
plt.figure(figsize=(10, 6))
sns.scatterplot(x='Temperature(蚓)', y='Rented Bike Count', data=df, alpha=0.5)
```

```
plt.title('Rented Bike Count vs. Temperature(蚓)')
plt.xlabel('Temperature(蚓)')
plt.ylabel('Rented Bike Count')
plt.show()
```

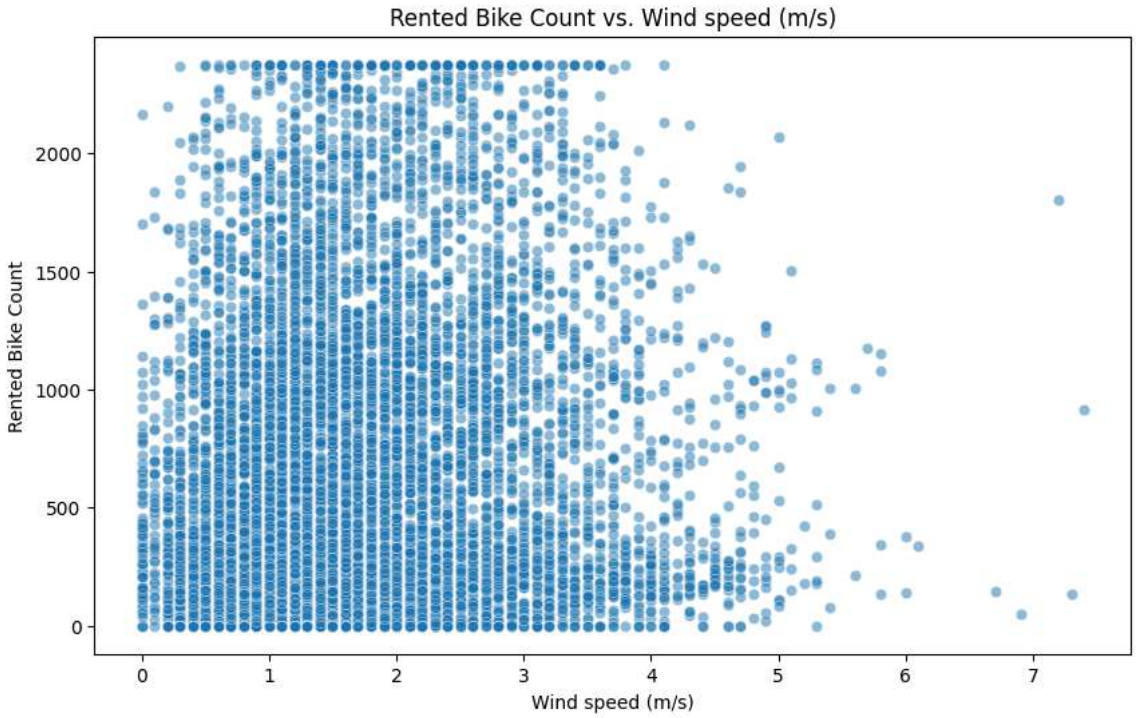
```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 34451 (\N{CJK UNIFIED IDEOGRAPH-8693}
  fig.canvas.print_figure(bytes_io, **kw)
```



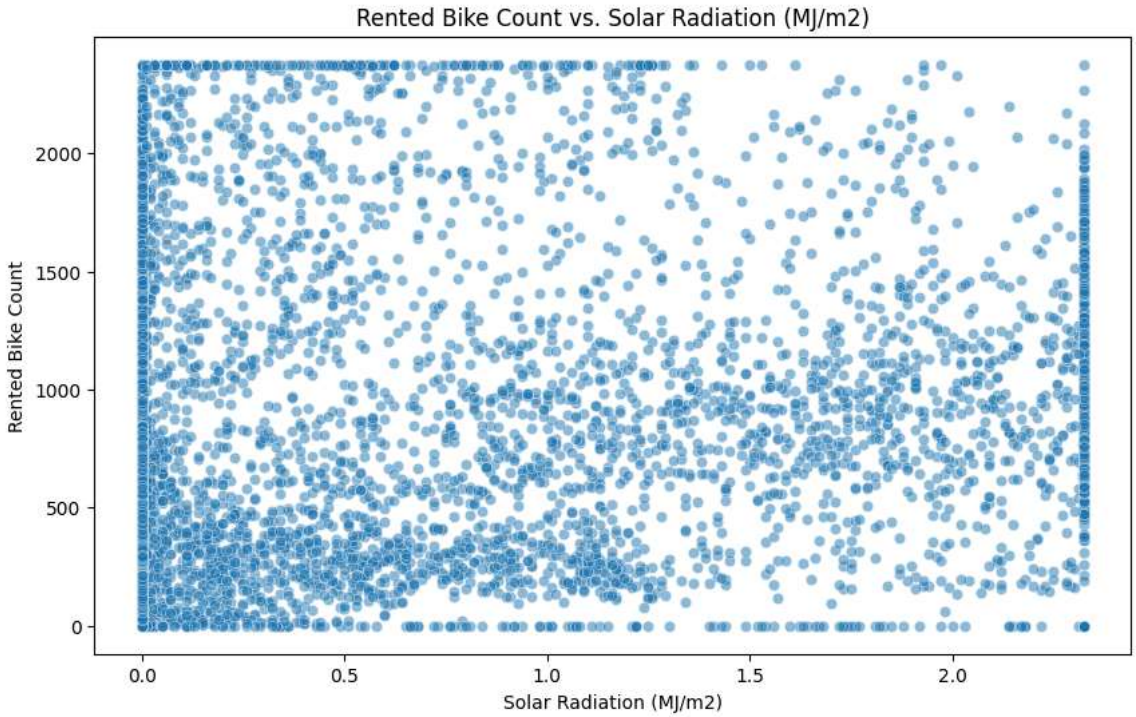

```
# 4. Rented Bike Count vs. Humidity
plt.figure(figsize=(10, 6))
sns.scatterplot(x='Humidity(%)', y='Rented Bike Count', data=df, alpha=0.5)
plt.title('Rented Bike Count vs. Humidity(%)')
plt.xlabel('Humidity(%)')
plt.ylabel('Rented Bike Count')
plt.show()
```



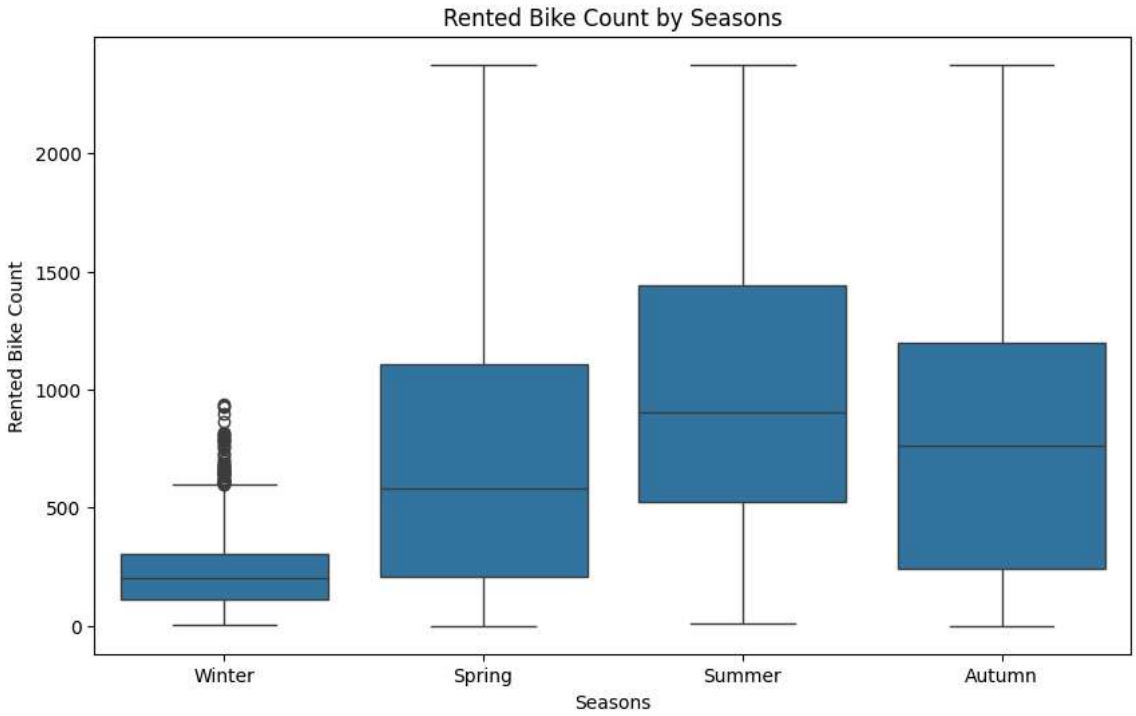
```
# 5. Rented Bike Count vs. Wind speed
plt.figure(figsize=(10, 6))
sns.scatterplot(x='Wind speed (m/s)', y='Rented Bike Count', data=df, alpha=0.5)
plt.title('Rented Bike Count vs. Wind speed (m/s)')
plt.xlabel('Wind speed (m/s)')
plt.ylabel('Rented Bike Count')
plt.show()
```



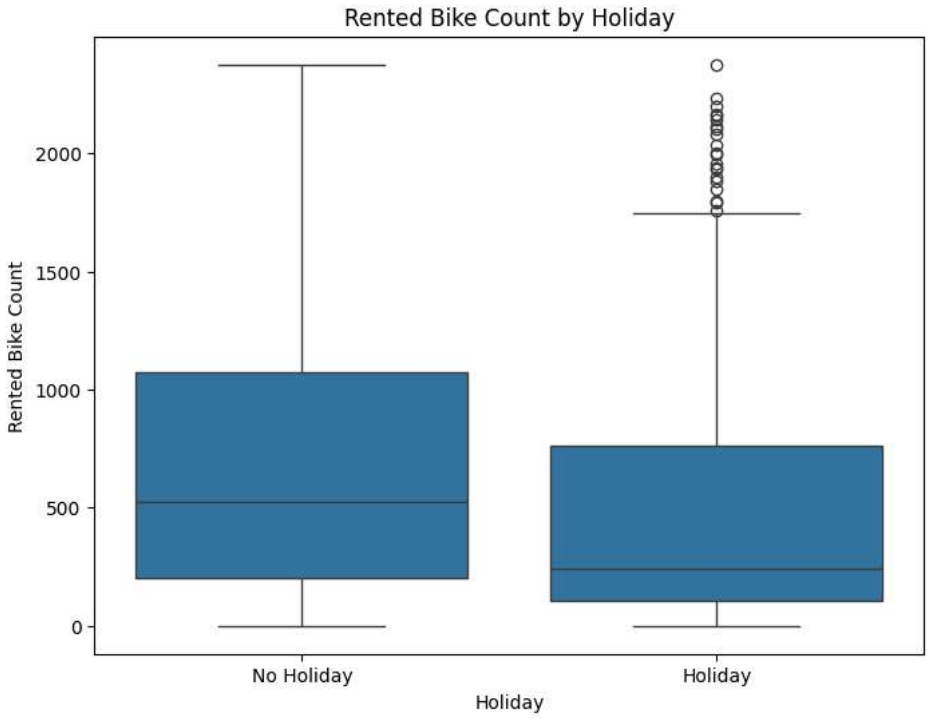

```
# 6. Rented Bike Count vs. Solar Radiation
plt.figure(figsize=(10, 6))
sns.scatterplot(x='Solar Radiation (MJ/m2)', y='Rented Bike Count', data=df, alpha=0.5)
plt.title('Rented Bike Count vs. Solar Radiation (MJ/m2)')
plt.xlabel('Solar Radiation (MJ/m2)')
plt.ylabel('Rented Bike Count')
plt.show()
```



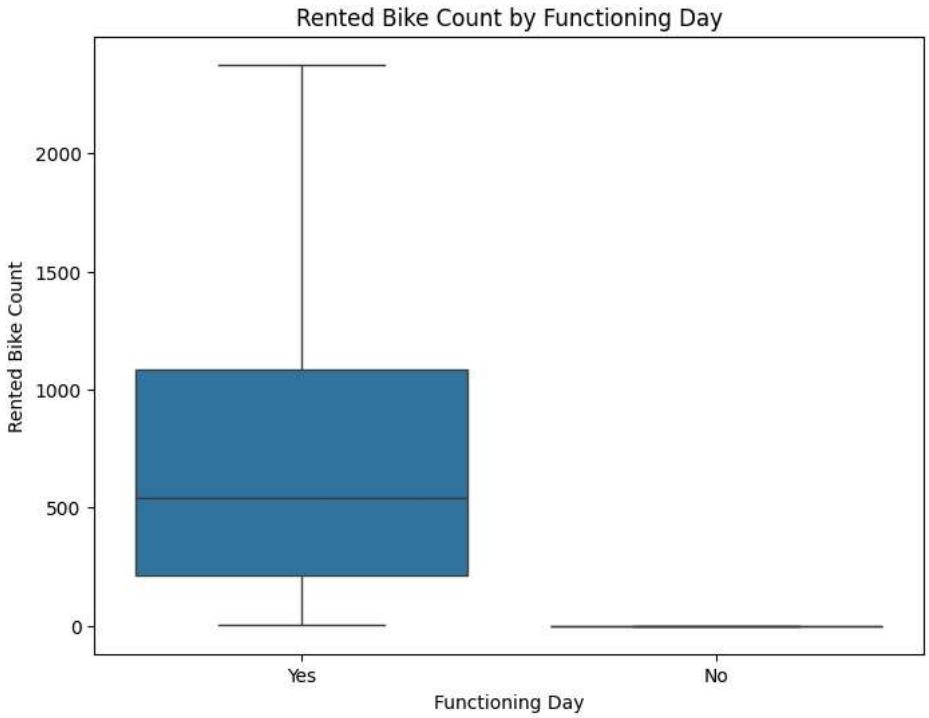
```
# 7. Rented Bike Count by Seasons
plt.figure(figsize=(10, 6))
sns.boxplot(x='Seasons', y='Rented Bike Count', data=df)
plt.title('Rented Bike Count by Seasons')
plt.xlabel('Seasons')
plt.ylabel('Rented Bike Count')
plt.show()
```



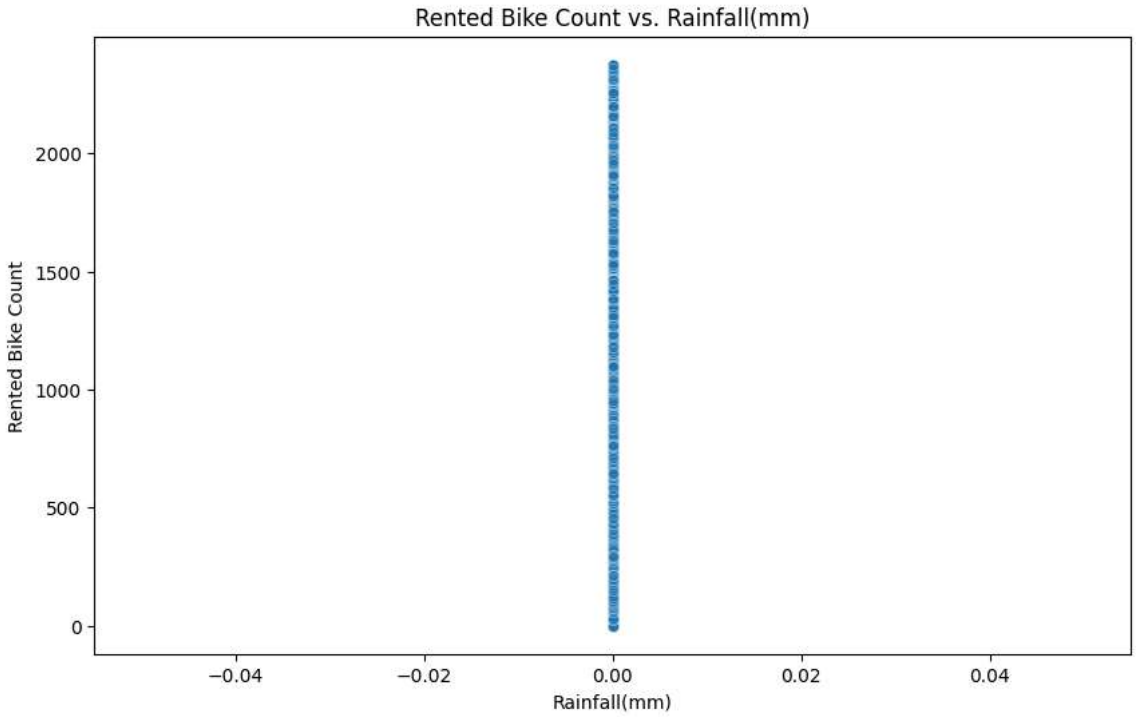

```
# 8. Rented Bike Count by Holiday
plt.figure(figsize=(8, 6))
sns.boxplot(x='Holiday', y='Rented Bike Count', data=df)
plt.title('Rented Bike Count by Holiday')
plt.xlabel('Holiday')
plt.ylabel('Rented Bike Count')
plt.show()
```



```
# 9. Rented Bike Count by Functioning Day
plt.figure(figsize=(8, 6))
sns.boxplot(x='Functioning Day', y='Rented Bike Count', data=df)
plt.title('Rented Bike Count by Functioning Day')
plt.xlabel('Functioning Day')
plt.ylabel('Rented Bike Count')
plt.show()
```




```
# 10. Rented Bike Count vs Rainfall
plt.figure(figsize=(10, 6))
sns.scatterplot(x='Rainfall(mm)', y='Rented Bike Count', data=df, alpha=0.5)
plt.title('Rented Bike Count vs. Rainfall(mm)')
plt.xlabel('Rainfall(mm)')
plt.ylabel('Rented Bike Count')
plt.show()
```



Feature Engineering: Creating New Time-Based Features

Let's extract additional features from the 'Date' column to capture temporal patterns that might influence bike rental counts. These new features will include:

- **Year**
- **Month**
- **Day of Week**
- **Day of Year**

```
# Ensure 'Date' column is in datetime format
df_encoded['Date'] = pd.to_datetime(df_encoded['Date'])

# Extract new time-based features
df_encoded['Year'] = df_encoded['Date'].dt.year
df_encoded['Month'] = df_encoded['Date'].dt.month
df_encoded['Day_of_Week'] = df_encoded['Date'].dt.dayofweek # Monday=0, Sunday=6
df_encoded['Day_of_Year'] = df_encoded['Date'].dt.dayofyear

print("New time-based features created.")
display(df_encoded.head())
```

```
New time-based features created.
```

	Date	Rented Bike Count	Hour	Temperature(蚓)	Humidity(%)	Wind speed (m/s)	Visibility (10m)	Dew point temperature(蚓)	Solar Radiation (MJ/m2)	Rainfall(mm)	Snowf (
0	2017-12-01	-0.709536	-1.661325	-1.513957	-1.042483	0.458476	0.925871	-1.659605	-0.682824	0.0	
1	2017-12-01	-0.789297	-1.516862	-1.539074	-0.993370	-0.892561	0.925871	-1.659605	-0.682824	0.0	
2	2017-12-01	-0.838748	-1.372399	-1.580936	-0.944257	-0.699556	0.925871	-1.667262	-0.682824	0.0	
3	2017-12-01	-0.944033	-1.227936	-1.597680	-0.895144	-0.796059	0.925871	-1.659605	-0.682824	0.0	
4	2017-12-01	-0.990294	-1.083473	-1.580936	-1.091596	0.554978	0.925871	-1.736177	-0.682824	0.0	

Model Training: Linear Regression, Decision Tree, and Random Forest

Now that our data is preprocessed and new features have been engineered, let's prepare it for training by splitting it into training and testing sets. We will then train and evaluate three different regression models: Linear Regression, Decision Tree Regressor, and Random Forest Regressor.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Define features (X) and target (y)
# Drop 'Date' column as it's no longer needed after extracting temporal features
X = df_encoded.drop(columns=['Date', 'Rented Bike Count'])
y = df_encoded['Rented Bike Count']

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Data split into training and testing sets.")
print(f"X_train shape: {X_train.shape}, y_train shape: {y_train.shape}")
print(f"X_test shape: {X_test.shape}, y_test shape: {y_test.shape}")
```

```
Data split into training and testing sets.
X_train shape: (7008, 18), y_train shape: (7008,)
X_test shape: (1752, 18), y_test shape: (1752,)
```

1. Linear Regression Model

```
# Initialize and train Linear Regression model
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

# Make predictions
y_pred_linear = linear_model.predict(X_test)

# Evaluate the model
mse_linear = mean_squared_error(y_test, y_pred_linear)
r2_linear = r2_score(y_test, y_pred_linear)

print(f"Linear Regression - Mean Squared Error: {mse_linear:.4f}")
print(f"Linear Regression - R-squared: {r2_linear:.4f}")
```

```
Linear Regression - Mean Squared Error: 0.4631
Linear Regression - R-squared: 0.5373
```

2. Decision Tree Regressor Model

```
# Initialize and train Decision Tree Regressor model
decision_tree_model = DecisionTreeRegressor(random_state=42)
decision_tree_model.fit(X_train, y_train)

# Make predictions
y_pred_tree = decision_tree_model.predict(X_test)

# Evaluate the model
mse_tree = mean_squared_error(y_test, y_pred_tree)
r2_tree = r2_score(y_test, y_pred_tree)

print(f"Decision Tree Regressor - Mean Squared Error: {mse_tree:.4f}")
print(f"Decision Tree Regressor - R-squared: {r2_tree:.4f}")
```

```
Decision Tree Regressor - Mean Squared Error: 0.1607
Decision Tree Regressor - R-squared: 0.8395
```

3. Random Forest Regressor Model

Model Performance Comparison

```
from sklearn.metrics import mean_absolute_error
import numpy as np

# --- Linear Regression ---
rmse_linear = np.sqrt(mse_linear)
mae_linear = mean_absolute_error(y_test, y_pred_linear)

print(f"Linear Regression:\n  RMSE: {rmse_linear:.4f}\n  MAE: {mae_linear:.4f}\n  R-squared: {r2_linear:.4f}\n")

# --- Decision Tree Regressor ---
rmse_tree = np.sqrt(mse_tree)
mae_tree = mean_absolute_error(y_test, y_pred_tree)

print(f"Decision Tree Regressor:\n  RMSE: {rmse_tree:.4f}\n  MAE: {mae_tree:.4f}\n  R-squared: {r2_tree:.4f}\n")

# --- Random Forest Regressor ---
rmse_rf = np.sqrt(mse_rf)
mae_rf = mean_absolute_error(y_test, y_pred_rf)

print(f"Random Forest Regressor:\n  RMSE: {rmse_rf:.4f}\n  MAE: {mae_rf:.4f}\n  R-squared: {r2_rf:.4f}\n")
```

```
Linear Regression:
  RMSE: 0.6805
  MAE: 0.5209
  R-squared: 0.5373

Decision Tree Regressor:
  RMSE: 0.4008
  MAE: 0.2085
  R-squared: 0.8395

Random Forest Regressor:
  RMSE: 0.2741
```

```
MAE: 0.1541
R-squared: 0.9249
```

Feature Importance Analysis

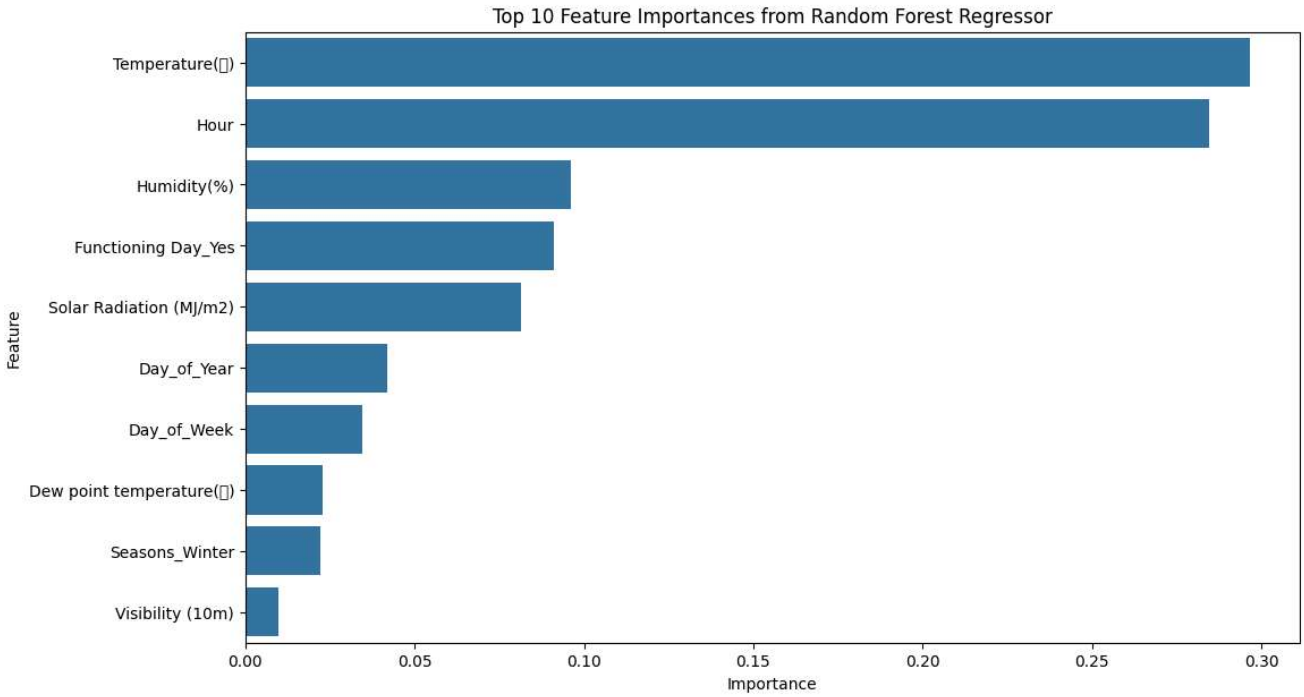
To understand which factors most significantly influence 'Rented Bike Count', let's analyze the feature importances from our best-performing model, the Random Forest Regressor. This will help us identify the key drivers for bike rentals.

```
# Get feature importances from the Random Forest model
feature_importances = random_forest_model.feature_importances_

# Create a pandas Series for better visualization
features_df = pd.DataFrame({'Feature': X.columns, 'Importance': feature_importances})
features_df = features_df.sort_values(by='Importance', ascending=False)

print("Top 5 Features Impacting Rented Bike Count:")
display(features_df.head(5))

# Optional: Visualize feature importances
plt.figure(figsize=(12, 7))
sns.barplot(x='Importance', y='Feature', data=features_df.head(10))
plt.title('Top 10 Feature Importances from Random Forest Regressor')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.show()
```

```
Top 5 Features Impacting Rented Bike Count:
```

	Feature	Importance
1	Temperature(蚓)	0.296509
0	Hour	0.284674
2	Humidity(%)	0.096140
13	Functioning Day_Yes	0.091045
6	Solar Radiation (MJ/m2)	0.081401

```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 34451 (\N{CJK UNIFIED IDEOGRAPH-8693}
  fig.canvas.print_figure(bytes_io, **kw)
```



```
# Initialize and train Random Forest Regressor model
random_forest_model = RandomForestRegressor(n_estimators=100, random_state=42)
random_forest_model.fit(X_train, y_train)

# Make predictions
y_pred_rf = random_forest_model.predict(X_test)

# Evaluate the model
mse_rf = mean_squared_error(y_test, y_pred_rf)
r2_rf = r2_score(y_test, y_pred_rf)

print(f"Random Forest Regressor - Mean Squared Error: {mse_rf:.4f}")
print(f"Random Forest Regressor - R-squared: {r2_rf:.4f}")
```

```
Random Forest Regressor - Mean Squared Error: 0.0751
Random Forest Regressor - R-squared: 0.9249
```